EXP NO:

HAMMING CODE

Aim:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature

Error correction at Data Link Layer:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from sender to the receiver

CODE:

```
def sender():
    data = input("Enter text: ")
    # convert text to binary
    binary = ""
    for ch in data:
        binary += format(ord(ch), '08b')

    print("Binary data:", binary)
    # find number of parity bits
    m = len(binary)
    r = 0
    while 2**r < m + r + 1:
        r += 1
    n = m + r
    code = ['0'] * (n+1)

    # put data bits
    j = 0
    for i in range(1, n+1):
        if i & (i-1):
```

```
# 12 bit Hamming code
# Position: 1 2 3 4 5 6 7 8 9 10 11 12
#           r1 r2 d1 r4 d2 d3 d4 r8 d5 d6 d7 d8
#
code = [0] * 13

# Put data bits.
code[3] = d[0]
code[5] = d[1]
code[6] = d[2]
code[7] = d[3]
code[9] = d[4]
code[10] = d[5]
code[11] = d[6]
code[12] = d[7]

# Calculate r1
code[1] = str(
    int(code[3]) ^ int(code[5]) ^ int(code[7]) ^
    int(code[9]) ^ int(code[11])
)

# calculate r2
code[2] = str(
    int(code[3]) ^ int(code[6]) ^ int(code[7]) ^
    int(code[10]) ^ int(code[11])
)

# calculate r4
code[4] = str(
    int(code[5]) ^ int(code[6]) ^ int(code[7]) ^
    int(code[12])
)

# Calculate r8
code[8] = str(
```

```
        int(code[9] ^ int(code[10]) ^
        int(code[11] ^ int(code[12])
    )
    # Add 12 bits to channel
    for i in range(1, 13):
        channel += code[i]

print("Hamming code:", channel)
# Save Hamming code in channel.txt
with open("channel.txt", "w") as f:
    f.write(channel)
print("Hamming code saved in channel.txt")
```

Receiver:

```
# r1, r2, r4 and r8 are redundancy bits
# Read data from channel.
with open("channel.txt", "r") as f:
    received = f.read()
print("Received data:", received)
corrected = ""
# Process 12 bits at a time
for x in range(0, len(received), 12):
    c = ['0'] + list(received[x:x+12])
# check r1
r1 = (
    int(c[1]) ^ int(c[3]) ^ int(c[5]) ^ int(c[7]) ^
    int(c[9]) ^ int(c[11])
)
# check r2
```

```
r2 = ( #
    int(c[2]) ^ int(c[3]) ^ int(c[6]) ^ int(c[7]) ^
    int(c[12])
    )

#r4
r4 = (
    int(c[4]) ^ int(c[5]) ^ int(c[6]) ^
    int(c[7]) ^ int(c[12])
    )

#Check r8
r8 = (
    int(c[4]) ^ int(c[5]) ^ int(c[6]) ^
    int(c[7]) ^ int(c[12])
    )

# Find error position
error = r1*1 + r2*2 + r4*4 + r8*8
if error == 0:
    print("No error detected.");
else:
    print("Error detected at position:", error)

    # corred the error
    if c[error] == '0'
        c[error] = '1'
    else:
        c[error] = '0'
    print("Error corrected")

# Remove redundancy bits
corrected += (
    c[3]+c[5]+c[6]+c[7]+c[9]+c[10]+c[11]+c[12])
)
```

```
#convert binary to text
output = ""
for x in range(0, len(corrected), 8):
    byte = corrected[x:x+8]
    if len(byte) == 8
        output += chr(int(byte, 2))

print("Original text", output)
```

output:

Enter the text: hello

Binary data : 01101000011001010110110001101111

Hamming code : 010011011000100111000101000011001100000011001100110111001111

Hamming code send in channel.txt

Received data: 010001011000100111000101000011001100000011001100110111001111

Error detected at position: 5

Error corrected

Result:

Thus the hamming code is implemented and the output is obtained.